

CachePool: Many-core cluster of customizable, lightweight scalar-vector PEs for irregular L2 data-plane workloads

Integrated Systems Laboratory (ETH Zürich)

Zexin Fu, Diyou Shen

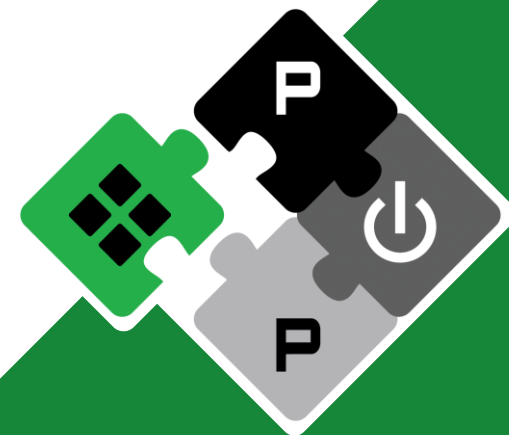
zexifu, dishen@iis.ee.ethz.ch

Alessandro Vanelli-Coralli
Luca Benini

avanelli@iis.ee.ethz.ch
lbenini@iis.ee.ethz.ch

PULP Platform

Open Source Hardware, the way it should be!



@pulp_platform



pulp-platform.org



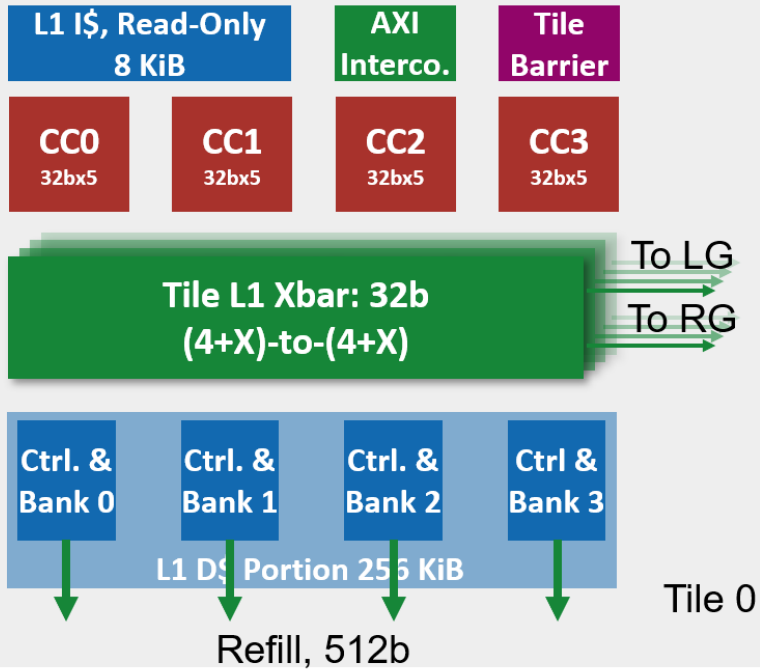
youtube.com/pulp_platform



Hardware Development (Cluster)



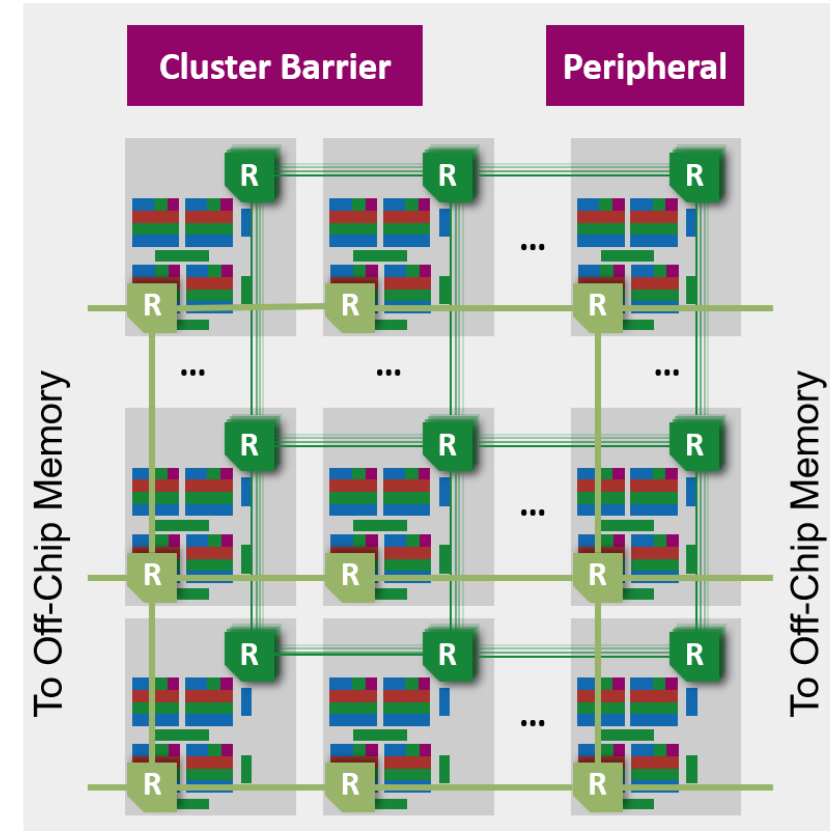
Tile



Group



Cluster



Hardware Development (Cluster)



- **Performance Monitor**

- Improve monitor to generate better report for performance checking
- RTL-only, cannot be used for synthesis

- **NoC Visualization**

- Visualize the NoC traffic for performance analysis
- Use vis4mesh tool

- **Partial Barrier Support**

- Add partial barrier into multi-group
- Two partial barrier scheme supported (can be used together):
 - Selected tiles participates in barrier => based on memory-mapped register
 - Selected cores within a tile participated in barrier => based on write data (no HW cost)



Hardware Development (Cluster)



```
*****
*** Session 3 (kernel): cycles [19475, 20185] ***
*****
***      Snitch Scalar Core      ***
Inst Fetch Accepted Count:      615
Inst Fetch AVG Serving Cycles:  1.15
Scalar LSU Load Req-to-Commit Count: 12
Scalar LSU Load AVG Req-to-Commit Cyc: 113.33
Stall Cycles Total:      656
Stall Cycles (No Valid Instr):  94
Stall Cycles (ITLB Miss):      0
Stall Cycles (RAW Hazard):     100
Stall Cycles (LSU Port Busy):   0
Stall Cycles (Accelerator Port): 462
Stall Cycles (Fence Drain):     0

VLSU Load Req-to-Commit Count:  64
VLSU Load AVG Req-to-Commit Cyc: 10.97

***      Spatz Controller Utilization      ***
VLSU:
VLSU Active (not accurate):      2
VLSU Num Instructions:           2
VLSU Stall Cycles:               0
VLSU WR Stalls:                  0.00
VFU:
VFU Active (not accurate):       3
VFU Num Instructions:            3
VFU Stall Cycles:               0
VFU WR Stalls:                  0.00
VSLDU:
VSLDU Active (not accurate):     0
VSLDU Num Instructions:          0
VSLDU Stall Cycles:             0
VSLDU WR Stalls:                0.00

***      Spatz VLSU Utilization      ***
Number of VRF Valid Cycles:      64
Number of VRF Transaction Counts: 64
VRF W Utilization:              100.00
VRF W AVG Cycles:               1.00
Number of Mem Valid Cycles:     64
Number of Mem Transaction Counts: 64
Mem Utilization:                100.00
```

Header report for performance checking

thesis

Performance analysis

Group

supported (can be used together):

er => based on memory-mapped register

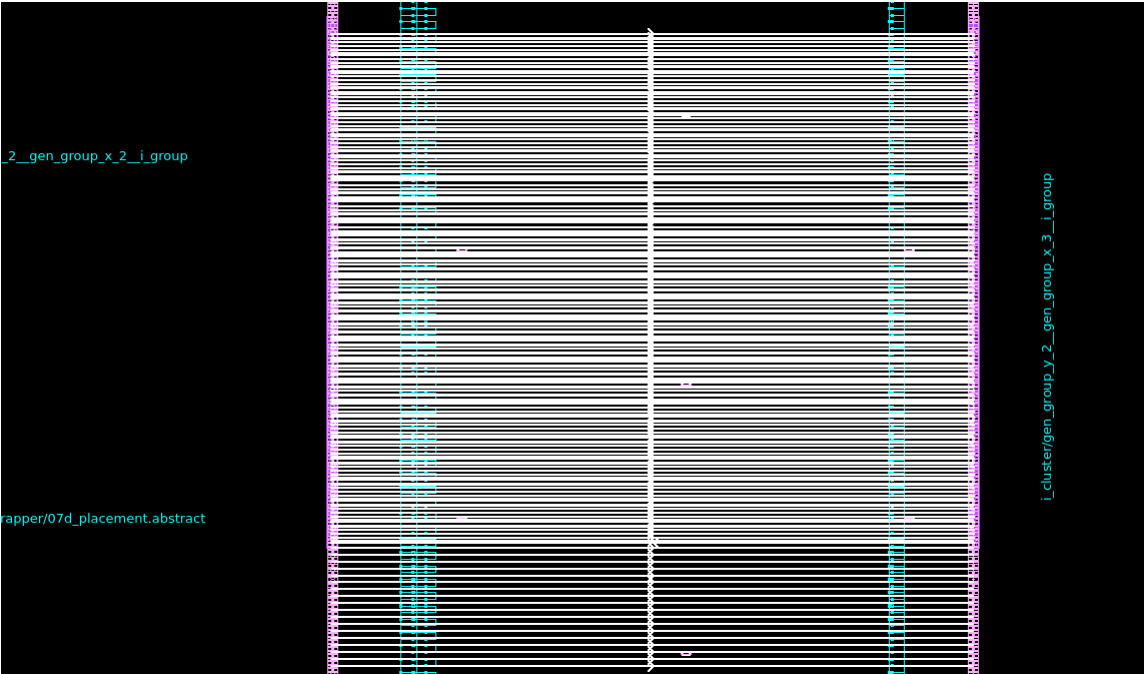
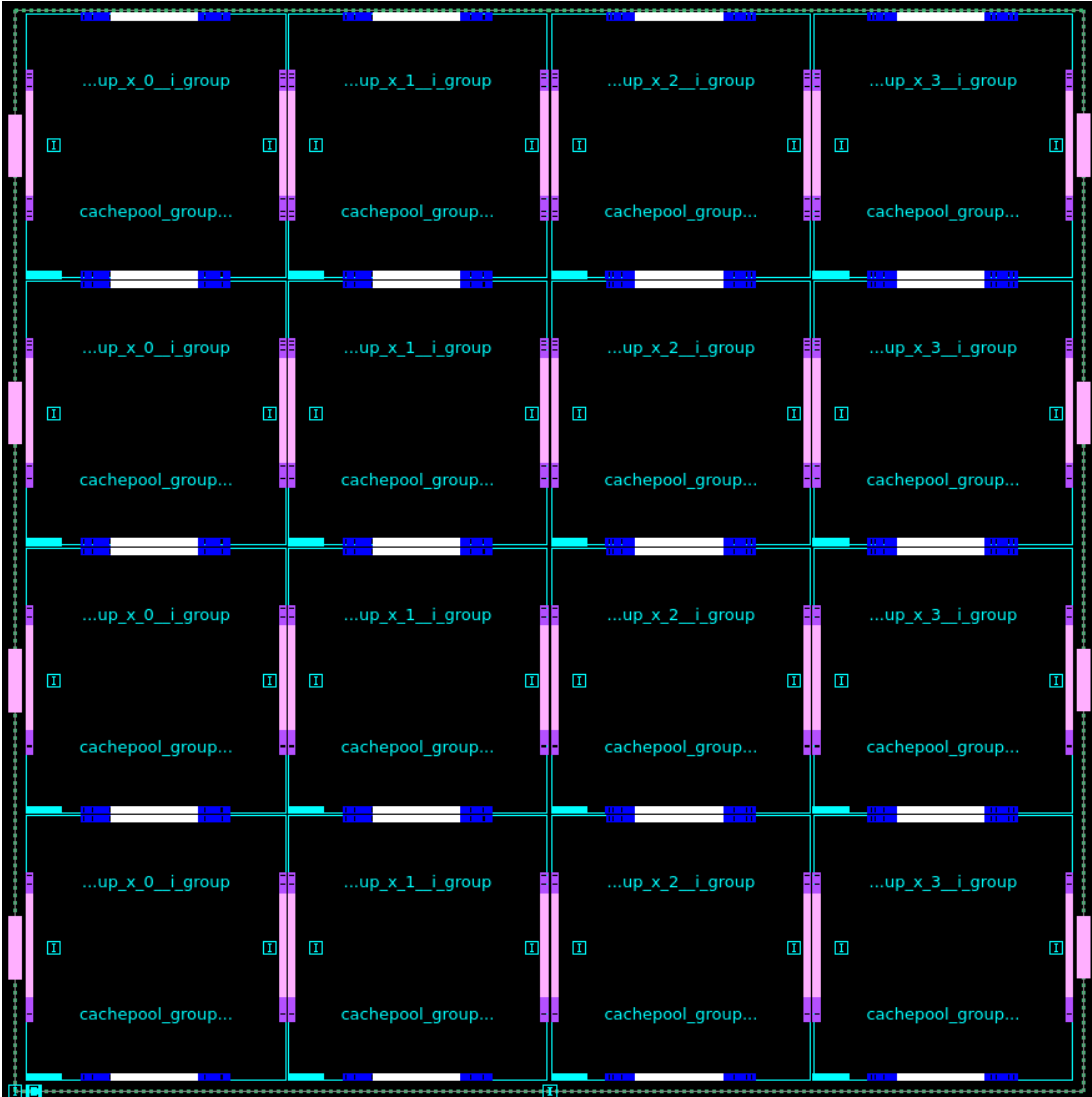
computed in barrier => based on write data (no HW cost)



Hardware Development (Cluster)



Physical Design (Multi-Group)

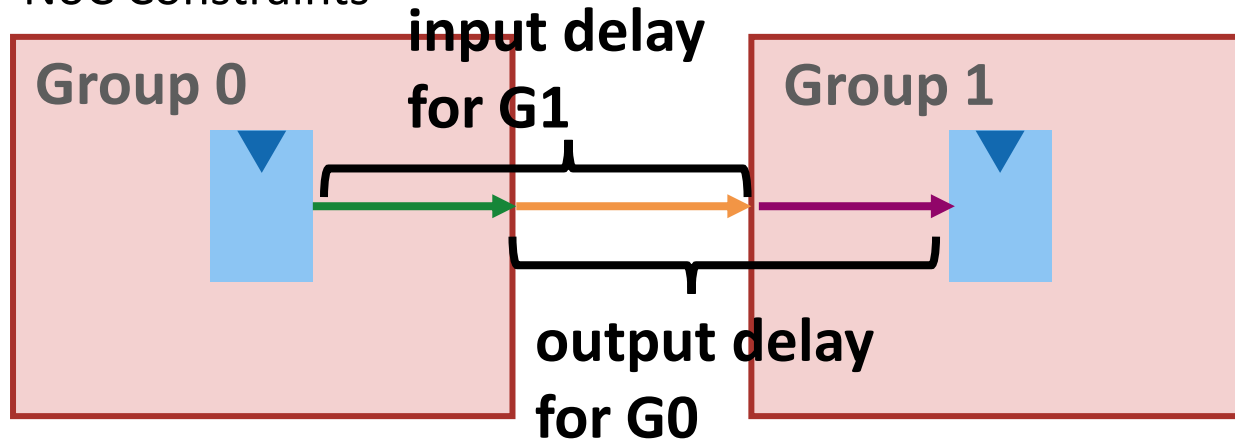


Physical Design (Multi-Group)



- **Group-Level**

- NoC Constraints



- 1st iteration shows some timing issues on peripheral paths
 - Applied cuts onto them (not performance critical)
- 2nd iteration under going

- **Cluster-Level**

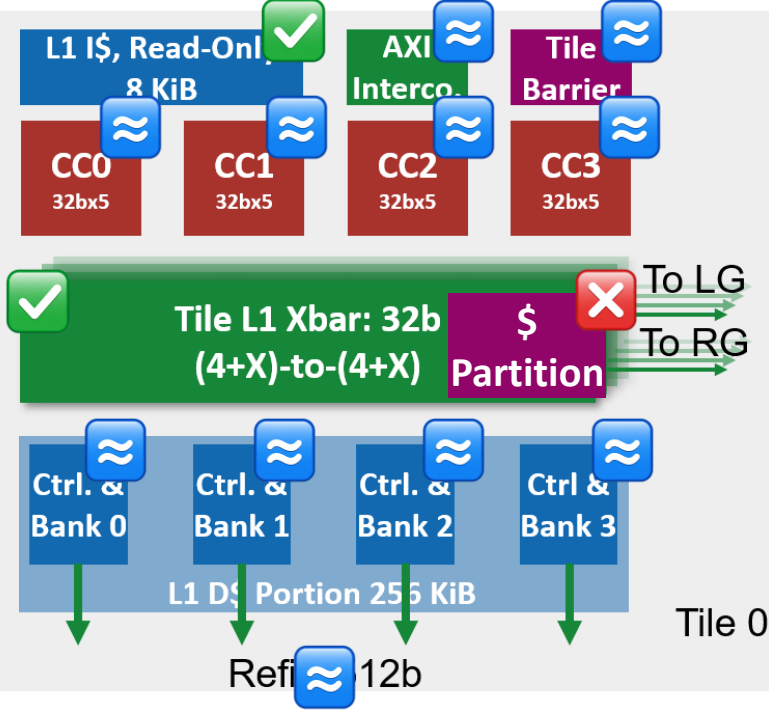
- FP, PG and pin placement done
- Now waiting for group level to continue placement (need IO timing info)



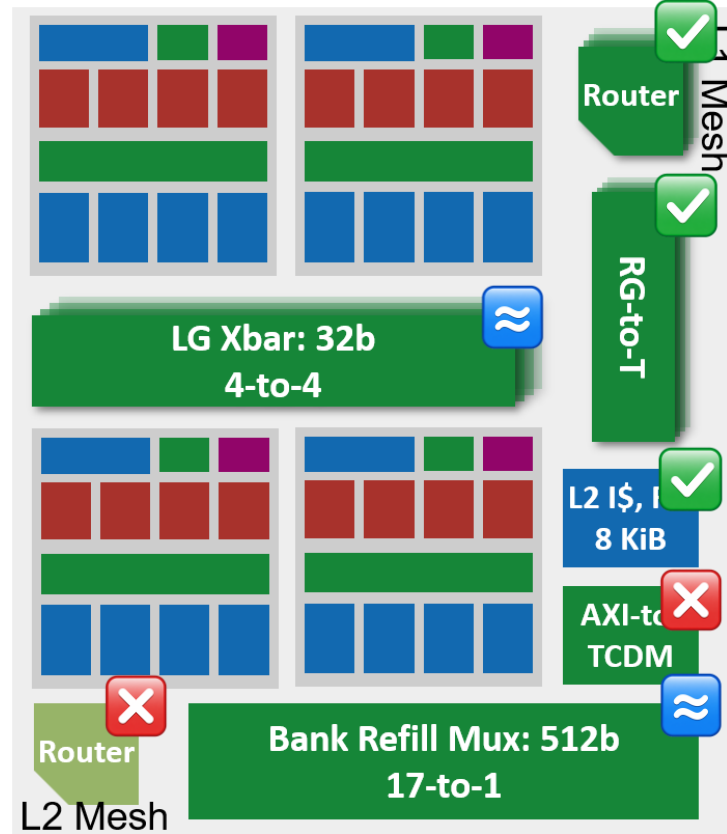
GVSoc Modeling Progress



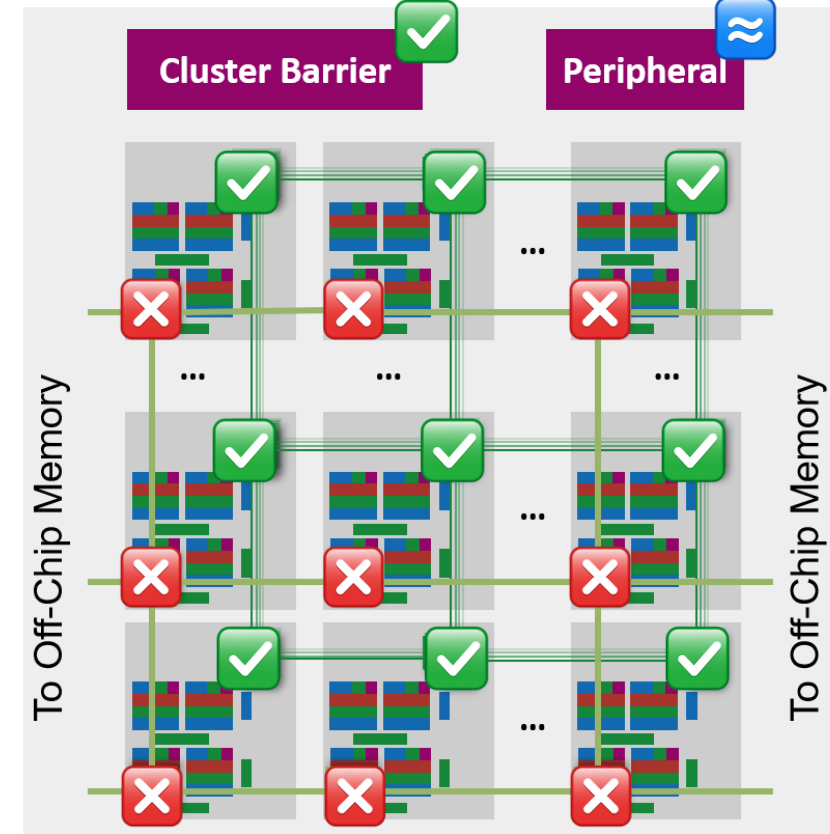
Tile



Group



Cluster



Done, the same logic as RTL



In place, with approximate model



Not implemented yet



GVSoc Modeling Progress



- **Model bugs fixed**

- **Barrier and AMO**

- The hardware barrier was mapped to the wrong address, so it never blocked and broke cross-core sum
 - The atomic write resp result was written to the wrong buffer, cause spin-lock livelock

- **Private stack:** All 16 cores shared one stack, overwriting each other's frames (crash + broke prints)

- **Cache and memory access**

- The vector unit marked memory bursts "done" at issue time, so consumers could read data before it actually arrived → commit bursts at issue + real latency.
 - A second reader of a line still being refilled got an instant fake "hit" on a not-installed line (early data race) → keep the line "pending" and make followers wait for the refill.
 - The backing memory had zero latency, so every cache miss was nearly free → use DRAMsys



GVSoc Modeling Progress



• Results

- Now all 8 CI test kernels are passed:

• fdotp-32b_M8192	✓ pass	→	✓ pass
• fdotp-32b_M32768	✓ pass	→	✓ pass
• gemv-opt_M512_N128_K32	✓ pass	→	✓ pass
• fmatmul-32b_M32_N32_K32	✓ pass	→	✓ pass
• fft-32b_M1024_N16	✗ wrong result	→	✓ pass
• load-store_M16	✗ hang	→	✓ pass
• RLC_M1_N1350_K10	✗ fail	→	✓ pass
• byte-enable	✗ fail	→	✓ pass
• spin-lock	✗ livelock	→	✓ pass

- Performance calibration is ongoing



RLC kernel multi-user use cases dev



- The kernel used to have **one** RLC entity (one UE) with one global state struct; now it has an **array of N entities** and every packet is routed to **its** entity

Component	BEFORE — single user only	AFTER — N users
RLC context	One global <i>rlc_ctx</i> containing both queues, SN state, polling state, and statistics	<i>rlc_ctx</i> [NUM_USERS]: one independent context per UE
List locks	Two global locks shared by all cores	Per-user to-send and wait-ACK locks, distributing contention
Sequence numbers	One global <i>vtNext</i> sequence-number stream	Independent <i>rlc_ctx[u].vtNext</i> stream for each RLC entity
Packet routing	Descriptor <i>user_id</i> exists but is ignored	<i>user_id</i> selects the target RLC context and queue
User count	One user	Compile-time NUM_USERS, derived from ACTIVE_USER_NUMBER



RLC kernel flow change



Single-user

Producers

One shared to-send
linked list

Consumers
(pop, assemble, push)

One shared wait-
wait-ACK linked list

ACK
processing

Multi-user

to-send linked list

Wait-ACK linked list

Producers

New pkg
routed by user_id

User 0
User 1
User 2
User 3
...
User 47

Consumer 0

Consumer 1

Consumer 2

...

User 0

User 1

User 2

User 3

...

User 47

ACK
processing

Currently use core 0 to
scan all users and do ACK

Each consumer handles
its assigned user



RLC kernel multi-user use cases dev



- Tested run in **GVSoc** model and **RTL** with small hw config
 - Single group hardware config, 16 cores
 - 48 users, 300pkg, 800B PDUs
 - GVSoc: P2/C2, P4/C4, P2/C8, P4/C8 pass
 - Next step is run with larger config, and report and compare the performance number
 - RTL: P2/C2,P4/C4, P2/C8 pass; P4/C8 fails
 - Need to debug the rtl simulation

	Kernel run cycle	Kernel run cycle		
SW config	GVSoc	RTL	Diff	Throughput (Gbps)
P2/C2	538635	530059	1.6%	3.56
P2/C8	505146	457988	10.3%	3.80
P4/C4	301139	316688	-4.9%	6.38
P4/C8	250210	fail		7.67



Next Step



- **Diyou Side**

- Physical design exploration for multi-group cluster => help to determine the BW we can have
 - First PD run looks very positive => we can increase BW between groups
- Rebase onto main (folded cache work) and merge
- Multi-Snitch cores sharing one Spatz
 - Plan
Add additional Snitch into the current CC (what we have done in a different project)
 - Some needed additions
 - 1) Snitch inside the tile should all connect to the Spatz, using muxing to select the host
 - 2) Software and runtime support to reduce the coding complexity (some helper functions)

- **Zexin Side**

- GVSoC development and calibration under multi-group config
- SW development for multi-user, debug under multi-group config



Timeline



		2025					2026							
		11	12	1	2	3	4	5	6	7	8	9	10	11
WP1 ManyCore Architecture	Multi-Tile Scaling [RTL] Complete													
	4/16-Group Scaling [GVSoC + RTL] Ongoing													
	Interco. Optimization (multi-group) [RTL] Ongoing													
	Timing optimization for NoC [RTL] Ongoing by colleague													
	NoC Topology Exploration [RTL, GVSoC] Ongoing by student													
WP2 Cache	Cache partitioning [RTL] Complete													
	WT-L1 by master student [RTL] Complete, Extending Ongoing													
	Folded Cache [GVSoC + RTL] Complete, Opt Ongoing													

Timeline



		2025						2026										
			11	12	1	2	3	4	5	6	7	8	9	10	11			
WP3 VPE	Shared vector core [RTL + GVSoC] Ongoing																	
WP4 Benchmark	Single-user kernel optimization [SW] Ongoing																	
	Multi-user and more scenarios [SW]																	
Report	Final Report and Benchmark																	

L1 Access Latency



- **Problem: increased L1 access latency when scaling up**
- **Solutions**
 - **(Ongoing, RTL & Physical) Hardware optimization**
Further optimize the interconnection and cache controller to reduce access latency on existing architecture
=> Retiming the current interconnection, remove unnecessary cuts in interco and cache
=> Target to reduce 1-2 cycles (~5 cyc) for hit-to-use latency
Currently has reduced to 6 cycles:
Core => Xbar => AMO => Ctrl (2) => AMO => Core
 - **(Complete, RTL) Cache partitioning**
Partition the shared L1 to keep data localized to avoid remote access latency
 - **(Complete with some problems, RTL & Physical) WT-L1**
Explore and evaluate a private WT L1 for each core
=> Performance degrade on single tile
=> Needs more work to support larger system



L1 Access Latency



- **Problem: large NoC latency (~28 cyc based on previous exploration)**
- **Solutions**
 - **(Ongoing, Physical) Reduce hop latency**
Explore the timing under 7nm technology to see the possibility of reducing per-hop latency to 1 cycle
=> Reduce the latency by 1/2
 - **(Planned, Physical) Cut clk-domain (optional)**
If 1-cyc is not possible, explore to operate the NoC under a higher frequency than cores (e.g. 1.5GHz)
=> Effectively cut the latency by 1/3
 - **(Planned, GVSoc & Physical) NoC Topology**
Explore different NoC topologies
=> Torus to reduce the diameter of NoC by 1/2, reduce longest latency by 1/2



PE-Related



- **Problem: Improve scalar core performance**
- **Solutions**
 - **(Planned, RTL & GVSoc) Shared vector core**
Explore two Snitch sharing one Spatz configuration to improve scalar performance
Software complexity needs to be considered, especially on the possible conflicting use of register file.
Discussed Solution: critical section



RLC Workload



- **Problem: Current RLC model cannot reveal all properties of the kernel**
- **Solutions**
 - **(Ongoing, SW) Kernel optimization**
Adapt the current model to better represent the real cases, e.g. do some operations on the data. Further cooperation with HQ side to develop a more complete and better model.
 - **(Planned, SW) Kernel development**
Implement the missing use cases for multi-user



Thank you!

Q&A

